

PART XV

DYNAMIC PROGRAMMING

Complete In-Depth Edition

State, transition, and the thinking process — not just the code

Part XIV ended with two broken greedy strategies — coin change on arbitrary denominations, and 0/1 knapsack via ratio-greedy. Both failed for the same underlying reason: an early locally-best choice closed off a globally-better combination that wasn't visible yet. Dynamic programming exists exactly to fix this failure mode: instead of committing to one choice per step, it systematically explores every relevant choice, but avoids the exponential blow-up of brute-force recursion by reusing the answers to subproblems it has already solved. This Part builds that idea from first principles — overlapping subproblems, optimal substructure, memoization vs. tabulation, and the state/transition/base-case framework — and then applies it across every major DP family: 1D and 2D DP, the full knapsack family, string DP, sequence DP, interval DP, tree DP, bitmask DP, and digit DP, finishing with the space and state-compression optimizations that make DP solutions fast enough for real constraints.

Contents of Part XV

- 1. What Is Dynamic Programming?
- 2. The DP Design Framework: How to Identify and Build a Solution
- 3. 1D DP: Fibonacci, Climbing Stairs, House Robber, Coin Change
- 4. 2D DP: Grid Paths and Obstacle Grids
- 5. The Knapsack Family
- 6. String DP: LCS, Longest Common Substring, Edit Distance
- 7. Sequence DP: Longest Increasing Subsequence and Variants
- 8. Interval DP: Matrix Chain Multiplication and Beyond
- 9. Tree DP
- 10. Bitmask DP
- 11. Digit DP
- 12. DP Optimization: Space, State Compression, and Prefix Tricks
- 13. Recognizing a DP Problem
- 14. Practice Drills
- Part XV Revision, Master Cheat Sheet, and Interview Questions

Chapter 1 — What Is Dynamic Programming?

1.1 The one-sentence definition

In one sentence: dynamic programming solves a problem by breaking it into overlapping subproblems, solving each distinct subproblem exactly once, and reusing those answers instead of recomputing them — turning an exponential brute-force search into a polynomial one.

This directly continues Part V, Chapter 4's observation about naive recursive Fibonacci: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ recomputes the same smaller values enormous numbers of times, giving $O(2^n)$ time despite there being only $n+1$ distinct subproblems ($\text{fib}(0)$ through $\text{fib}(n)$). Dynamic programming is precisely the discipline of noticing this waste and eliminating it.

1.2 The two required properties

Property	Meaning	Fibonacci example
Overlapping subproblems	The same smaller subproblem is needed more than once while solving the bigger problem	$\text{fib}(3)$ is needed by both $\text{fib}(5)$ and $\text{fib}(4)$
Optimal substructure	An optimal (or correct) solution to the problem can be built directly from optimal solutions to its subproblems	$\text{fib}(n)$ is exactly $\text{fib}(n-1) + \text{fib}(n-2)$ — no other combination is needed

WARNING — A problem without overlapping subproblems doesn't need DP

If a recursive breakdown never revisits the same subproblem twice (e.g. merge sort's recursion tree — every node represents a distinct sub-range), memoization has nothing to save — you'd just be adding bookkeeping overhead to an already-optimal divide-and-conquer algorithm. Always check for genuine overlap before reaching for DP.

1.3 Memoization (top-down)

```
unordered_map<int, long long> memo;
long long fib(int n) {
    if (n <= 1) return n;
    if (memo.count(n)) return memo[n]; // already solved – reuse it
    return memo[n] = fib(n - 1) + fib(n - 2); // solve once, cache it
}
```

Memoization keeps the natural recursive structure of the brute force, adding a cache check at the top of the function and a cache write before returning. This is usually the fastest way to convert a brute-force recursive solution you already have into a working DP — write the brute force first (Part II's discipline), confirm it's correct, then memoize it.

1.4 Tabulation (bottom-up)

```
long long fibTab(int n) {
    if (n <= 1) return n;
    vector<long long> dp(n + 1);
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2]; //
    build up from the base case
    return dp[n];
}
```

	Memoization (top-down)	Tabulation (bottom-up)
Direction	Starts from the target, recurses down to base cases	Starts from base cases, builds up to the target
Code shape	Recursive function + cache	Iterative loop + array
Computes unneeded states?	No — only states actually reached by recursion	Sometimes — fills every state in the loop's range
Recursion/stack overhead	Yes — function call overhead, risk of stack overflow for deep recursion	No — pure iteration
Easier to derive from a brute force	Yes — usually a direct, mechanical conversion	Requires figuring out a valid computation order up front

KEY IDEA — Which to pick

Start by writing the brute force, then memoize it — this is almost always the fastest path to a first working DP solution and rarely requires new thinking beyond adding a cache. Convert to tabulation afterward specifically when you need to avoid recursion's stack-depth risk (Part V, Chapter 4) or want the extra speed and space-optimization opportunities tabulation enables (Chapter 12).

1.5 The four-part vocabulary used throughout this Part

- 1 State — the minimal set of parameters that uniquely identifies a subproblem (for Fibonacci, just n).
- 2 Transition — the formula/rule expressing a state's answer in terms of other (smaller/simpler) states' answers ($dp[i] = dp[i-1] + dp[i-2]$).
- 3 Base case — the smallest states, answered directly without recursion ($dp[0]=0$, $dp[1]=1$).

- 4 Answer extraction — which state (or combination of states) holds the final answer to the original problem ($dp[n]$).

Every DP problem in this Part is presented by explicitly naming these four things first, before showing code — this is deliberate: identifying them correctly is solving the problem; the code is just a direct translation afterward.

Chapter 2 — The DP Design Framework: How to Identify and Build a Solution

2.1 The systematic process

- 1 Confirm overlapping subproblems and optimal substructure are actually present (Chapter 1).
- 2 Define the state in words first — what does $dp[\dots]$ MEAN, in a full sentence — before writing any index notation.
- 3 Define the transition — how does this state's answer relate to smaller states?
- 4 Define the base case(s) — the smallest states that can be answered directly.
- 5 Decide the order of computation — for tabulation, every state a transition depends on must already be computed; for memoization, this is handled automatically by recursion.
- 6 Identify where the final answer lives — a single state, or the best/sum over a set of states.
- 7 Compute the complexity as (number of distinct states) $\times$ (work per transition) — Part V, Chapter 9.
- 8 Implement — memoize the brute force, or tabulate directly.
- 9 Test against small hand-traced cases and, if unsure of correctness, stress-test (Part II) against the brute force.

KEY IDEA — The most common beginner mistake in this entire process

Jumping straight to writing `dp[i][j] = ...` without first stating, in a complete sentence, exactly what `dp[i][j]` represents. If you cannot state the meaning of a state in one sentence, you do not yet understand the problem well enough to write its transition correctly — this mirrors Part II's insistence on restating a problem formally before coding.

2.2 A fully worked example of the process: climbing stairs

Problem: you can climb 1 or 2 steps at a time; how many distinct ways are there to reach step n ?

- 1 Overlapping subproblems? Yes — the number of ways to reach step 5 is needed by both the computation for step 6 and step 7.
- 2 State (in words): `dp[i]` = the number of distinct ways to reach step i .
- 3 Transition: to reach step i , your last move was either a 1-step from $i-1$, or a 2-step from $i-2$. So `dp[i] = dp[i-1] + dp[i-2]`.
- 4 Base case: `dp[0] = 1` (one way to 'reach' the ground: do nothing), `dp[1] = 1` (one way: a single 1-step).
- 5 Order of computation: increasing i , since `dp[i]` needs only smaller indices.
- 6 Answer: `dp[n]`.
- 7 Complexity: n states, $O(1)$ work each $\rightarrow O(n)$ time, $O(n)$ space (reducible to $O(1)$, Chapter 12).

```
int climbStairs(int n) {
    if (n <= 1) return 1;
    vector<int> dp(n + 1);
    dp[0] = 1; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}
```

NOTE — Notice the shape

This is structurally identical to Fibonacci — the state, transition, and complexity are the same, only the problem's story differs. Recognizing 'this is secretly Fibonacci's shape' is a direct application of Part II's 'strip the story away' habit, now applied specifically to DP.

Chapter 3 — 1D DP: Fibonacci, Climbing Stairs, House Robber, Coin Change

3.1 House Robber

Problem: given house values, maximize total value robbed such that no two adjacent houses are both robbed.

- State: $dp[i]$ = the maximum value obtainable considering only houses $0..i$.
- Transition: at house i , either skip it ($dp[i-1]$) or rob it and add to the best from $i-2$ ($dp[i-2] + a[i]$) — take the better option: $dp[i] = \max(dp[i-1], dp[i-2] + a[i])$.
- Base case: $dp[0] = a[0]$; $dp[1] = \max(a[0], a[1])$.
- Answer: $dp[n-1]$.

```
int houseRobber(vector<int> &a) {
    int n = a.size();
    if (n == 1) return a[0];
    vector<int> dp(n);
    dp[0] = a[0];
    dp[1] = max(a[0], a[1]);
    for (int i = 2; i < n; i++) dp[i] = max(dp[i - 1], dp[i - 2] + a[i]);
    return dp[n - 1];
}
```

Dry run: a = [2, 7, 9, 3, 1]

i	a[i]	dp[i] = max(dp[i-1], dp[i-2]+a[i])	dp[i]
0	2	base	2
1	7	max(2,7)	7
2	9	max(7, 2+9)	11
3	3	max(11, 7+3)	11
4	1	max(11, 11+1)	12

Final answer: 12 (rob houses at indices 0, 2, 4: $2+9+1=12$).

WARNING — Common mistake: confusing this with the exchange-argument greedy from Part XIV

There is no valid greedy rule here — 'always rob the larger of any adjacent pair' fails on inputs like [2, 1, 1, 9], where skipping the early houses entirely is required to reach 9. This is exactly the subsequence problem type flagged back in Part III, Chapter 5, as needing DP rather than a simple scan.

3.2 Coin change: minimum number of coins

Problem: given coin denominations and a target amount, find the minimum number of coins to make that amount (or determine it's impossible) — this is precisely the problem Part XIV, Chapter 8 showed greedy fails on for arbitrary denominations.

- State: $dp[x]$ = the minimum number of coins to make amount x .
- Transition: for each coin $c \leq x$, $dp[x] = \min(dp[x], dp[x-c] + 1)$ — try using one of coin c and solve the rest optimally.
- Base case: $dp[0] = 0$ (zero coins needed for amount zero).
- Answer: $dp[\text{target}]$, or 'impossible' if it remains at infinity.

```

int coinChange(vector<int> &coins, int target) {
    vector<int> dp(target + 1, INT_MAX);
    dp[0] = 0;
    for (int x = 1; x <= target; x++) {
        for (int c : coins) {
            if (c <= x && dp[x - c] != INT_MAX) {
                dp[x] = min(dp[x], dp[x - c] + 1);
            }
        }
    }
    return dp[target] == INT_MAX ? -1 : dp[target];
}

```

Dry run: coins = {1, 3, 4}, target = 6 (the Part XIV counterexample)

x	Best coin choice	dp[x]
0	base	0
1	1: dp[0]+1	1
2	1+1: dp[1]+1	2
3	coin 3: dp[0]+1	1
4	coin 4: dp[0]+1	1
5	coin 1: dp[4]+1, or coin 4: dp[1]+1	2
6	coin 3: dp[3]+1 = 2	2

dp[6] = 2, matching the optimal 3+3 answer from Part XIV — exactly where greedy's 4+1+1 = 3 coins was wrong. This is the direct fix DP provides: by keeping the best answer for every smaller amount, it never commits early to a choice that turns out suboptimal.

States	Work per transition	Total complexity
O(target)	O(number of coins)	O(target * number of coins)

Chapter 4 — 2D DP: Grid Paths and Obstacle Grids

4.1 Unique paths in a grid

Problem: count the number of distinct paths from the top-left to the bottom-right of an $n \times m$ grid, moving only right or down.

- State: $dp[i][j]$ = the number of distinct paths from $(0,0)$ to (i,j) .
- Transition: to reach (i,j) , the last move was either from above $(i-1,j)$ or from the left $(i,j-1)$: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$.
- Base case: $dp[0][j] = 1$ for all j (only one way along the top row); $dp[i][0] = 1$ for all i (only one way down the left column).
- Answer: $dp[n-1][m-1]$.

```
int uniquePaths(int n, int m) {
    vector<vector<int>> dp(n, vector<int>(m, 1)); // top row and
    left column are all 1 by construction
    for (int i = 1; i < n; i++)
        for (int j = 1; j < m; j++)
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    return dp[n - 1][m - 1];
}
```

Dry run: 3x3 grid

Row \ Col	0	1	2
0	1	1	1
1	1	2	3
2	1	3	6

6 paths from top-left to bottom-right of a 3x3 grid — matches the closed-form combinatorial answer $C(4,2) = 6$ (Part XVII will cover this combinatorial connection directly).

4.2 Grid paths with obstacles

Modification: some cells are blocked and cannot be stepped on. The transition is identical except a blocked cell contributes $dp = 0$ (no paths pass through it) regardless of what its neighbors computed.

```
int uniquePathsWithObstacles(vector<vector<int>> &grid) { // 1 =
obstacle, 0 = open
    int n = grid.size(), m = grid[0].size();
    vector<vector<long long>> dp(n, vector<long long>(m, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 1) { dp[i][j] = 0; continue; }
            if (i == 0 && j == 0) { dp[i][j] = 1; continue; }
            long long fromAbove = (i > 0) ? dp[i - 1][j] : 0;
            long long fromLeft = (j > 0) ? dp[i][j - 1] : 0;
            dp[i][j] = fromAbove + fromLeft;
        }
    }
    return dp[n - 1][m - 1];
}
```

4.3 Minimum path sum

Problem: find the path from top-left to bottom-right minimizing the sum of values along the way (same movement rules).

```
int minPathSum(vector<vector<int>> &grid) {
    int n = grid.size(), m = grid[0].size();
    vector<vector<int>> dp(n, vector<int>(m));
    dp[0][0] = grid[0][0];
    for (int j = 1; j < m; j++) dp[0][j] = dp[0][j-1] + grid[0][j];
    for (int i = 1; i < n; i++) dp[i][0] = dp[i-1][0] + grid[i][0];
    for (int i = 1; i < n; i++)
        for (int j = 1; j < m; j++)
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]);
    return dp[n-1][m-1];
}
```

KEY IDEA — The shape shared by every grid DP in this chapter

State = position (i,j); transition = combine the two (or more) cells that could have led here; base case = the first row and column, which only have one way in. Once this shape is recognized, every 'grid + movement rule + combine values' problem becomes a direct fill-in-the-template exercise.

States	Work per transition	Total complexity
$O(n * m)$	$O(1)$	$O(n * m)$

Chapter 5 — The Knapsack Family

5.1 0/1 Knapsack

Problem: n items, each with a weight and a value; a knapsack of capacity W ; maximize total value, each item usable at most once (this is the exact problem Part XIV's ratio-greedy failed on).

- State: $dp[i][w]$ = the maximum value achievable using only the first i items with capacity budget w .
- Transition: for item i , either skip it ($dp[i-1][w]$) or take it, if it fits ($dp[i-1][w - \text{weight}[i]] + \text{value}[i]$) — take the better option.
- Base case: $dp[0][w] = 0$ for all w (no items considered yet, no value possible).
- Answer: $dp[n][W]$.

```

int knapsack01(int W, vector<int> &weight, vector<int> &value) {
    int n = weight.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w]; // option 1: skip item i
            if (weight[i - 1] <= w) // option 2: take item i, if it
fits
                dp[i][w] = max(dp[i][w], dp[i - 1][w - weight[i -
1]] + value[i - 1]);
        }
    }
    return dp[n][W];
}

```

Dry run: the Part XIV counterexample — W=50, A(10,60), B(20,100), C(30,120)

Ratio-greedy took A+B for value 160; DP correctly considers every combination, including B+C (weight 50, value 220), and returns 220 — proving DP fixes exactly the failure mode Part XIV, Chapter 8 demonstrated.

States	Work per transition	Total complexity
$O(n * W)$	$O(1)$	$O(n * W)$

WARNING — Why this is called 'pseudo-polynomial'

The complexity $O(n*W)$ depends on the numeric VALUE of W , not just the number of items n . If W is up to 10^9 , this DP is far too slow despite looking polynomial on paper — always check the actual numeric range of capacity-like parameters against the operation budget (Part V, Chapter 1) before trusting a knapsack DP's speed.

5.2 Unbounded knapsack

Modification: each item can be used unlimited times. The only change from 0/1 knapsack is that taking item i can reuse row i itself (not $i-1$),

since the item remains available:

```
int unboundedKnapsack(int W, vector<int> &weight, vector<int>
&value) {
    int n = weight.size();
    vector<int> dp(W + 1, 0);
    for (int w = 1; w <= W; w++) {
        for (int i = 0; i < n; i++) {
            if (weight[i] <= w) dp[w] = max(dp[w], dp[w - weight[i]]
+ value[i]);
        }
    }
    return dp[W];
}
```

NOTE — The 0/1 vs. unbounded distinction, generalized

Whenever a DP allows reusing the same 'item' unlimited times, the transition references the SAME row/state currently being computed rather than a strictly smaller one — this is the single mechanical difference between the two knapsack variants, and it generalizes to any 'limited vs. unlimited supply' DP.

5.3 Subset sum

Problem: can a subset of the given numbers sum to exactly a target T? This is 0/1 knapsack with weight = value = each number, and asking only about feasibility rather than maximization.

```
bool subsetSum(vector<int> &a, int T) {
    vector<bool> dp(T + 1, false);
    dp[0] = true; // empty subset sums to 0
    for (int x : a) {
        for (int t = T; t >= x; t--) { // iterate DOWNWARD to avoid
reusing x twice (0/1, not unbounded)
            dp[t] = dp[t] || dp[t - x];
        }
    }
    return dp[T];
}
```

WARNING — Iterate the capacity loop DOWNWARD for 0/1 problems using a 1D array

This is one of the most common DP bugs. When the DP array is compressed to 1D (Chapter 12) for a 0/1 problem, the inner loop over capacity MUST go from high to low — going low to high would let the same item be 'reused' within a single pass, silently turning a 0/1 knapsack into an unbounded one. Unbounded knapsack (Section 5.2) is exactly the case where iterating upward is correct.

5.4 Partition equal subset sum

Problem: can the array be split into two subsets with equal sum? Directly reduces to subset sum: if the total sum is odd, impossible; otherwise, ask `subsetSum(a, totalSum / 2)` — if one half can be formed, the remaining elements automatically form the other equal half.

5.5 Target sum (assign +/- to reach a target)

Problem: assign a + or – sign to each number so the total equals a target S . Key algebraic transformation: let P be the sum of positively-signed numbers and N the sum of negatively-signed ones. $P - N = S$ and $P + N = \text{totalSum}$, so $P = (S + \text{totalSum}) / 2$ — the problem reduces to counting subsets that sum to exactly P , another

direct subset-sum variant.

```
int targetSumWays(vector<int> &a, int S) {
    int total = accumulate(a.begin(), a.end(), 0);
    if ((S + total) % 2 != 0 || S > total) return 0; // P must be a
    non-negative integer
    int P = (S + total) / 2;
    vector<int> dp(P + 1, 0);
    dp[0] = 1;
    for (int x : a)
        for (int t = P; t >= x; t--)
            dp[t] += dp[t - x];
    return dp[P];
}
```

KEY IDEA — The pattern across this whole chapter

Every knapsack-family problem is the same core transition (skip vs. take an item) wearing a different question on top — maximize value, check feasibility, or count ways. Recognizing 'this is knapsack-shaped' and then identifying which of these three questions is being asked determines the rest of the solution almost mechanically.

Chapter 6 — String DP: LCS, Longest Common Substring, Edit Distance

6.1 Longest Common Subsequence (LCS)

Problem: given two strings, find the length of the longest subsequence (Part III's precise definition — order preserved, gaps allowed) common to both.

- State: $dp[i][j]$ = the length of the LCS of the first i characters of s_1 and the first j characters of s_2 .

- Transition: if $s1[i-1] == s2[j-1]$, this matching character extends the LCS of the smaller prefixes: $dp[i][j] = dp[i-1][j-1] + 1$. Otherwise, take the best of skipping a character from either string: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
- Base case: $dp[0][j] = dp[i][0] = 0$ (an empty string has no common subsequence with anything).
- Answer: $dp[n][m]$.

```
int lcs(string &s1, string &s2) {
    int n = s1.size(), m = s2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[n][m];
}
```

Dry run: $s1 = \text{"ABCB DAB"}\text{, } s2 = \text{"BDCAB"}\text{ (partial table)}$

	""	B	D	C	A	B
""	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2

Continuing the full table yields $dp[7][5] = 4$ (one LCS: "BCAB").

6.2 Longest Common Substring (contrast with LCS)

Critical distinction (echoing Part III, Chapter 5): a substring must be contiguous, unlike a subsequence. The DP is nearly identical, with one crucial change: a mismatch resets the current run to 0 rather than carrying forward the best of two neighbors, because a substring cannot skip a character and remain contiguous.

```
int longestCommonSubstring(string &s1, string &s2) {
    int n = s1.size(), m = s2.size(), best = 0;
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
                best = max(best, dp[i][j]);
            }
            // else: dp[i][j] stays 0 – NO carrying forward, unlike
LCS
        }
    }
    return best;
}
```

WARNING — The single-line difference that changes the whole problem

Removing the 'else $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ ' line is what converts LCS into longest common SUBSTRING. Confusing these two is one of the most common DP mistakes precisely because the code looks almost identical — always re-confirm which of the two the problem is actually asking for.

6.3 Edit Distance (Levenshtein distance)

Problem: minimum number of insertions, deletions, and substitutions to transform string $s1$ into string $s2$.

- State: $dp[i][j]$ = the minimum edit distance between the first i characters of $s1$ and the first j characters of $s2$.

- Transition: if $s1[i-1] == s2[j-1]$, no edit needed here: $dp[i][j] = dp[i-1][j-1]$. Otherwise, take the best of three options: substitute ($dp[i-1][j-1]+1$), delete from $s1$ ($dp[i-1][j]+1$), or insert into $s1$ ($dp[i][j-1]+1$).
- Base case: $dp[0][j] = j$ (insert j characters); $dp[i][0] = i$ (delete i characters).
- Answer: $dp[n][m]$.

```
int editDistance(string &s1, string &s2) {
    int n = s1.size(), m = s2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) dp[i][j] = dp[i - 1][j - 1];
            else dp[i][j] = 1 + min({dp[i-1][j-1], dp[i-1][j],
dp[i][j-1]});
        }
    }
    return dp[n][m];
}
```

States	Work per transition	Total complexity
$O(n * m)$	$O(1)$	$O(n * m)$

Chapter 7 — Sequence DP: Longest Increasing Subsequence and Variants

7.1 LIS: the $O(n^2)$ DP solution

Problem: find the length of the longest strictly increasing subsequence of an array.

- State: $dp[i]$ = the length of the longest increasing subsequence that ENDS exactly at index i .
- Transition: $dp[i] = 1 + \max(dp[j])$ over every $j < i$ with $a[j] < a[i]$ (extend the best qualifying earlier subsequence); if no such j exists, $dp[i] = 1$ (the element alone).
- Base case: every $dp[i]$ starts at 1 (each element is a valid length-1 subsequence by itself).
- Answer: $\max(dp[i])$ over all i — NOT $dp[n-1]$, since the longest increasing subsequence need not end at the last element.

```
int lis(vector<int> &a) {
    int n = a.size();
    vector<int> dp(n, 1);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
    return *max_element(dp.begin(), dp.end());
}
```

Dry run: $a = [10, 9, 2, 5, 3, 7, 101, 18]$

i	a[i]	dp[i]
0	10	1
1	9	1
2	2	1
3	5	2 (from a[2]=2)
4	3	2 (from a[2]=2)
5	7	3 (from a[3]=5 or a[4]=3)
6	101	4 (from a[5]=7)
7	18	4 (from a[5]=7)

$\max(dp) = 4$ — one valid LIS is $[2, 5, 7, 101]$ or $[2, 3, 7, 101]$.

7.2 LIS: the $O(n \log n)$ improvement

Maintain an array `tails` where `tails[k]` holds the smallest possible tail value of any increasing subsequence of length $k+1$ found so far. For each new element, binary search (Part VI) for its position in `tails` and either extend or replace — this is a direct application of the improvement cycle from Part II: the $O(n^2)$ bottleneck was the inner linear scan for 'the best `dp[j]` with `a[j] < a[i]`', and maintaining `tails` in sorted order lets binary search replace it.

```
int lisFast(vector<int> &a) {
    vector<int> tails;
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) tails.push_back(x); // x extends the
longest subsequence so far
        else *it = x; // x gives a smaller tail for this length
    }
    return tails.size();
}
```

WARNING — `tails` is NOT necessarily a real subsequence from the array

`tails` only tracks the smallest tail value achievable for each length — it is a bookkeeping structure, not an actual increasing subsequence you could point to in the array. Reconstructing the actual LIS (not just its length) needs extra parent-pointer tracking, not just this array.

Approach	Time	Space
$O(n^2)$ DP	$O(n^2)$	$O(n)$
$O(n \log n)$ with binary search	$O(n \log n)$	$O(n)$

7.3 Variants built on the same state idea

Variant	Change from standard LIS
Longest non-decreasing subsequence	Use $\leq$ instead of $<$ in the comparison
Longest decreasing subsequence	Reverse the array, or flip the comparison direction
Maximum sum increasing subsequence	Transition becomes $dp[i] = a[i] + \max(dp[j] \text{ for } a[j] < a[i])$, answer is $\max(dp)$, not the count
Number of longest increasing subsequences	Track a parallel $count[i]$ array alongside $dp[i]$, summing counts whenever a tie for the best length is found

Chapter 8 — Interval DP: Matrix Chain Multiplication and Beyond

8.1 What makes a DP an "interval DP"

Interval DP problems have states indexed by a range $[i, j]$ (rather than a single index), and their transition tries every possible way to split the range into two parts, combining the best answer for each part. This is a direct structural generalization of the triangular-nested-loop shape from Part V, Chapter 3.

8.2 Matrix Chain Multiplication

Problem: given a chain of matrices to multiply together, choose the parenthesization (order of multiplication) minimizing the total number of scalar multiplications.

- State: $dp[i][j]$ = the minimum cost to multiply the chain of matrices from index i to j .
- Transition: try every split point k between i and j : $dp[i][j] = \min \text{ over } k \text{ of } (dp[i][k] + dp[k+1][j] + \text{cost of multiplying the resulting two combined matrices})$.

- Base case: $dp[i][i] = 0$ (a single matrix needs no multiplication).
- Answer: $dp[0][n-1]$.

```
int matrixChainOrder(vector<int> &dims) { // dims[i-1] x dims[i] is
the size of matrix i
    int n = dims.size() - 1; // number of matrices
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int len = 2; len <= n; len++) { // interval length,
smallest first
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k+1][j] +
dims[i]*dims[k+1]*dims[j+1];
                dp[i][j] = min(dp[i][j], cost);
            }
        }
    }
    return dp[0][n-1];
}
```

KEY IDEA — The computation-order rule for every interval DP

Notice the outer loop iterates by increasing INTERVAL LENGTH, not by increasing i or j directly. This guarantees $dp[i][k]$ and $dp[k+1][j]$ — both strictly shorter intervals than $[i,j]$ — are already computed before they're needed. This 'shortest intervals first' ordering is the standard, reusable computation order for every interval DP, not just this one problem.

States	Work per transition	Total complexity
$O(n^2)$ intervals	$O(n)$ split points to try	$O(n^3)$

8.3 The general interval DP template

```
for (int len = 2; len <= n; len++) {
    for (int i = 0; i + len - 1 < n; i++) {
        int j = i + len - 1;
        for (int k = i; k < j; k++) {
            dp[i][j] = combine(dp[i][j], dp[i][k], dp[k+1][j] /* +
cost of merging at k */);
        }
    }
}
```

NOTE — Where else this template applies

Burst Balloons, optimal binary search tree construction, and palindrome-partitioning-with-minimum-cuts problems all fit this exact same 'shortest intervals first, try every split point' shape — recognizing the template is more valuable than memorizing any single problem built on it.

Chapter 9 — Tree DP

9.1 The core idea: DP over a rooted tree via DFS

Tree DP computes, for every node, an answer that depends on the (already-computed) answers of its children — a DFS post-order traversal (Part XI covers traversals in full) naturally provides the correct computation order, since a node's children are always fully processed before the node itself returns.

9.2 Worked example: maximum independent set on a tree

Problem: select a subset of tree nodes, no two of which are directly connected by an edge, maximizing the count (or sum of values) selected — the tree generalization of House Robber (Chapter 3), since

a parent and child can't both be picked, just like adjacent houses.

- State: for each node u , two values: $dp[u][0]$ = best answer for the subtree rooted at u if u is NOT selected; $dp[u][1]$ = best answer if u IS selected.
- Transition: $dp[u][1] = value[u] + \text{sum over children } c \text{ of } dp[c][0]$ (if u is picked, no child can be picked). $dp[u][0] = \text{sum over children } c \text{ of } \max(dp[c][0], dp[c][1])$ (if u isn't picked, each child independently picks its own best option).
- Base case: a leaf node has $dp[leaf][1] = value[leaf]$, $dp[leaf][0] = 0$.
- Answer: $\max(dp[root][0], dp[root][1])$.

```
vector<vector<int>> adj;
vector<int> value;
vector<array<int,2>> dp;

void dfs(int u, int parent) {
    dp[u][1] = value[u];
    dp[u][0] = 0;
    for (int c : adj[u]) {
        if (c == parent) continue;
        dfs(c, u);
        dp[u][1] += dp[c][0]; // picking u forces skipping every
child
        dp[u][0] += max(dp[c][0], dp[c][1]); // skipping u lets each
child choose freely
    }
}
// Call dfs(root, -1), then answer = max(dp[root][0], dp[root][1]).
```

WARNING — Recursion depth on trees

As flagged in Part V, Chapter 4: a DFS on a path-shaped tree (essentially a linked list) has recursion depth $O(n)$, which can approach stack overflow territory for n in the hundreds of thousands. For very large trees, either convert to an explicit iterative post-order traversal (Part XI) or increase the stack size if the judge allows it.

States	Work per transition	Total complexity
$O(n)$ nodes x 2 (selected/not)	$O(\text{children of that node})$, summed = $O(n)$ total across the tree	$O(n)$

9.3 Rerooting technique (introduction)

Some problems need the answer computed for EVERY possible choice of root (e.g. 'for each node, what is the sum of distances to all other nodes'). Recomputing a full tree DP from scratch for each of the n possible roots would cost $O(n^2)$ total. Rerooting instead computes the answer for one root via a single DFS, then re-derives every other node's answer from its parent's answer in $O(1)$ via a second DFS pass — reusing the same 'total work bounded, not recomputed from scratch' principle behind amortized analysis (Part V, Chapter 7).

NOTE — Scope note

Full rerooting implementations are intricate and problem-specific; this introduction is meant to make the technique recognizable — 'compute once from an arbitrary root, then shift the root by re-deriving in $O(1)$ per neighbor' — so that a dedicated rerooting problem doesn't look like an unfamiliar category the first time it's encountered.

Chapter 10 — Bitmask DP

10.1 When to reach for bitmask DP

Whenever a problem's state naturally includes 'which subset of a small set of items ($n \leq \sim 20$) has been used/visited/chosen so far,' represent that subset as an integer bitmask (Part XVI covers the bit-manipulation mechanics in full) and use it as one dimension of the DP state — this is exactly the $n \leq 20 \rightarrow$ subset enumeration signal from Part II's constraints table, now combined with DP instead of plain brute-force

enumeration.

10.2 Worked example: Traveling Salesman Problem (TSP)

Problem: visit every city exactly once and return to the start, minimizing total travel distance. Brute force tries all $(n-1)!$ orderings — infeasible beyond $n \approx 10$.

- State: $dp[mask][i]$ = the minimum cost to have visited exactly the set of cities in mask, ending currently at city i .
- Transition: $dp[mask][i] = \min$ over every j in mask, $j \neq i$, of $dp[mask \text{ without } i][j] + dist[j][i]$ — arriving at i last, having just come from some other visited city j .
- Base case: $dp[\{0\}][0] = 0$ (starting at city 0, having visited only city 0).
- Answer: $\min$ over i of $dp[full \text{ mask}][i] + dist[i][0]$ (return to the start).

```

int tsp(vector<vector<int>> &dist) {
    int n = dist.size();
    vector<vector<int>> dp(1 << n, vector<int>(n, INT_MAX));
    dp[1][0] = 0; // mask with only city 0 visited, currently at
    city 0
    for (int mask = 1; mask < (1 << n); mask++) {
        for (int i = 0; i < n; i++) {
            if (!(mask & (1 << i)) || dp[mask][i] == INT_MAX)
                continue;
            for (int j = 0; j < n; j++) {
                if (mask & (1 << j)) continue; // j already visited
                int newMask = mask | (1 << j);
                dp[newMask][j] = min(dp[newMask][j], dp[mask][i] +
                dist[i][j]);
            }
        }
    }
    int fullMask = (1 << n) - 1, best = INT_MAX;
    for (int i = 0; i < n; i++)
        if (dp[fullMask][i] != INT_MAX)
            best = min(best, dp[fullMask][i] + dist[i][0]);
    return best;
}

```

States	Work per transition	Total complexity
$O(2^n * n)$	$O(n)$	$O(2^n * n^2)$

KEY IDEA — Why this beats brute force enormously

$O(2^n * n^2)$ for $n=18$ is about $18^2 * 2^{18} \approx 85$ million — comfortably fast. Brute force $(n-1)!$ for $n=18$ is about 3.5×10^{14} — utterly infeasible. Bitmask DP achieves this by collapsing the $(n-1)!$ distinct orderings down to only $2^n * n$ distinct states, since many different orderings that visit the same set and end at the same city are interchangeable for the purposes of future decisions — exactly optimal substructure (Chapter 1) applied to permutation-shaped problems.

Chapter 11 — Digit DP

11.1 What digit DP solves

Digit DP counts numbers within a range (typically $[0, N]$) satisfying some digit-based property (e.g. 'the digit sum is at most K ', 'no two adjacent digits are equal') without enumerating every number individually — essential when N can be up to 10^{18} , far too large to loop over directly.

11.2 The state: position, tight, and problem-specific info

- `pos` — which digit position (from the most significant) is currently being decided.
- `tight` — a boolean: are the digits chosen so far exactly equal to N 's digits up to this position (constraining what digits remain legal), or already strictly smaller (making every remaining digit choice 0-9 legal)?
- Extra state — whatever the specific property needs to track (e.g. running digit sum, whether a particular digit has been used yet).

```

// Count numbers in [0, N] whose digit sum is exactly K
string digits;
int K;
vector<vector<vector<int>>> memo; // memo[pos][sumSoFar][tight]

int solve(int pos, int sumSoFar, bool tight) {
    if (sumSoFar > K) return 0; // pruned: can never reach exactly K
    anymore
    if (pos == (int)digits.size()) return sumSoFar == K;
    if (memo[pos][sumSoFar][tight] != -1) return
memo[pos][sumSoFar][tight];
    int limit = tight ? (digits[pos] - '0') : 9;
    int result = 0;
    for (int d = 0; d <= limit; d++) {
        result += solve(pos + 1, sumSoFar + d, tight && (d ==
limit));
    }
    return memo[pos][sumSoFar][tight] = result;
}

int countWithDigitSum(long long N, int targetK) {
    digits = to_string(N);
    K = targetK;
    memo.assign(digits.size(), vector<vector<int>>(K + 1,
vector<int>(2, -1)));
    return solve(0, 0, true);
}

```

NOTE — Why 'tight' is the key insight

Once a chosen digit is strictly less than N's digit at that position, every following digit is free to be anything 0-9 without ever exceeding N — the remaining subtree becomes independent of N entirely, and its answer only depends on how many positions remain and the extra state. This is exactly why 'tight' collapses what looks like an astronomically large search space down to a manageable, memoizable one.

States	Work per transition	Total complexity
$O(\text{digits} * \text{possible sum values} * 2)$	$O(10)$ digit choices	$O(\text{digits} * K * 10)$

WARNING — Range queries: $\text{count}(N) - \text{count}(L-1)$

To answer 'how many qualifying numbers in $[L, R]$ ', compute $\text{countUpTo}(R) - \text{countUpTo}(L-1)$ rather than trying to parametrize the DP directly on a lower bound — this mirrors the exact prefix-sum-difference technique from Part III applied to a digit-DP counting function instead of an array sum.

Chapter 12 — DP Optimization: Space, State Compression, and Prefix Tricks

12.1 Space optimization: rolling arrays

Whenever a transition for row i only references row $i-1$ (not any older row), the DP only needs to keep the two most recent rows in memory, not the entire table — an application of the $O(n)$ auxiliary space vs. $O(1)$ tradeoff from Part V, Chapter 5.

```
// House Robber (Chapter 3), space-optimized from  $O(n)$  to  $O(1)$ 
int houseRobberOptimized(vector<int> &a) {
    int prev2 = 0, prev1 = 0;
    for (int x : a) {
        int current = max(prev1, prev2 + x);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

```
// 0/1 Knapsack, space-optimized from  $O(n \cdot W)$  to  $O(W)$  using the
// downward-iteration trick (Chapter 5.3)
int knapsack01Optimized(int W, vector<int> &weight, vector<int>
&value) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < (int)weight.size(); i++)
        for (int w = W; w >= weight[i]; w--)
            dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);
    return dp[W];
}
```

KEY IDEA — The general rolling-array rule

If row i 's transition only needs row $i-1$: keep two 1D arrays (or one, with careful downward iteration for 0/1-style problems, as in Chapter 5.3). If row i needs rows $i-1$ AND $i-2$ (like Fibonacci-shaped DPs): keep three variables/arrays. Reducing a dimension of the table to $O(1)$ or $O(k)$ previous rows is one of the highest-value, lowest-effort optimizations in competitive DP.

12.2 State compression beyond bitmasks

Bitmask DP (Chapter 10) is the most common form of state compression, but the underlying idea — replacing a large or unwieldy piece of state with a compact representation that still distinguishes every state the transition actually cares about — generalizes further: coordinate compression (Part IV, Chapter 13) compresses a large value range into a small rank; some DPs compress 'the multiset of remaining items' into just a count per distinct item type, when duplicates make individual identity irrelevant.

12.3 Prefix-sum optimization in DP transitions

When a transition is $dp[i] = \max/\text{sum}$ over a RANGE of previous states (e.g. $dp[i] = \max(dp[j])$ for j in $[i-k, i-1]$), computing that range naively for every i costs $O(k)$ each, $O(n \cdot k)$ total. If the range aggregation is a

sum, a prefix sum (Part III) answers it in $O(1)$ after $O(n)$ preprocessing; if it's a min/max over a SLIDING window specifically, a monotonic deque (Part IV's monotonic-stack idea, extended to a deque) answers it in amortized $O(1)$ per step.

```
// dp[i] = a[i] + max(dp[i-k..i-1]), using a monotonic deque for
// O(1) amortized range-max
deque<int> dq; // stores indices, dp values decreasing front-to-back
vector<long long> dp(n);
for (int i = 0; i < n; i++) {
    while (!dq.empty() && dq.front() < i - k) dq.pop_front(); // out
    of window
    long long best = dq.empty() ? 0 : dp[dq.front()];
    dp[i] = a[i] + best;
    while (!dq.empty() && dp[dq.back()] <= dp[i]) dq.pop_back(); //
    maintain decreasing order
    dq.push_back(i);
}
```

NOTE — This is the exact monotonic-stack proof from Part IV, reapplied

Each index enters and leaves the deque at most once across the whole run, giving $O(n)$ total — the same 'bounded total movement' argument used for the monotonic stack (Part IV) and sliding window (Part III), now optimizing a DP transition instead of a standalone array problem.

Chapter 13 — Recognizing a DP Problem

Clue	Likely DP family
"Count the number of ways to..."	Counting DP — transition sums contributions rather than taking a max/min
"Maximum/minimum value subject to a capacity/budget constraint"	Knapsack family
"Longest/shortest subsequence satisfying an order condition"	Sequence DP (LIS-shaped)
Two strings compared position by position	String DP (LCS/edit distance-shaped)
"Optimal way to split/parenthesize/partition a sequence"	Interval DP
A tree, with a per-node optimal choice depending on children	Tree DP
$n \leq \sim 20$, and a notion of 'which items have been used'	Bitmask DP
Counting/property questions over numbers up to 10^{18}	Digit DP
A greedy idea that has a small counterexample (Part XIV, Chapter 8)	Almost certainly needs DP instead

KEY IDEA — The single most reliable test

Can you name the STATE in one sentence, and does computing today's answer require trying multiple candidate 'last decisions' rather than one obviously-best one? If yes to both, it's DP. If a single locally-best choice is provably always safe (an exchange argument holds, Part XIV Chapter 2), it's greedy instead, and greedy will be faster and simpler.

Chapter 14 — Practice Drills

Problem 15.1 — Maximum subarray sum, DP-style (Kadane's, revisited) [Level 1]

Re-derive Kadane's algorithm (Part II) explicitly as a 1D DP: state, transition, base case, answer.

Approach: $dp[i]$ = the maximum subarray sum ENDING at i = $\max(a[i], dp[i-1] + a[i])$; base case $dp[0] = a[0]$; answer = max over all $dp[i]$. This is the same algorithm, now understood as DP rather than an ad-hoc greedy-looking trick.

Complexity: $O(n)$ time, $O(1)$ space with rolling optimization.

Problem 15.2 — Minimum number of squares summing to N [Level 2]

Find the minimum number of perfect squares (1, 4, 9, 16, ...) that sum to N.

Hint 1: This is unbounded knapsack in disguise — each square can be used unlimited times.

Key observation: $dp[n] = 1 + \min(dp[n - s])$ over every perfect square $s \leq n$.

Complexity: $O(N * \sqrt{N})$ — $\sqrt{N}$ distinct squares up to N.

Problem 15.3 — Palindrome partitioning — minimum cuts [Level 4]

Find the minimum number of cuts needed to partition a string so every resulting piece is a palindrome.

Hint 1: Precompute `isPalindrome[i][j]` for every substring first (itself a small interval DP), then run a 1D DP over cut positions.

Key observation: $dp[i] = \text{minimum cuts for the prefix ending at } i = \min \text{ over every } j < i \text{ where } s[j+1..i] \text{ is a palindrome, of } dp[j] + 1.$

Complexity: $O(n^2)$ for the palindrome table, $O(n^2)$ for the cuts DP $\rightarrow O(n^2)$ total.

Problem 15.4 — Egg drop (classic interval-style DP) [Level 4]

With E eggs and F floors, find the minimum number of trials needed in the worst case to find the critical floor (the highest floor from which an egg doesn't break).

Hint 1: State: `dp[eggs][floors]` or reframe as `dp[trials][eggs] = \text{max floors distinguishable}`.

Key observation: Reframing the state as 'max floors solvable with a given number of trials and eggs' instead of the naive direction is a well-known trick that reduces the complexity substantially — a reminder that the FIRST state definition you try isn't always the most efficient one.

Complexity: $O(E * F)$ naively, $O(E * \sqrt{F})$ or $O(E \log F)$ with the reframed state.

Part XV — Master Revision

Must Remember

- DP requires both overlapping subproblems and optimal substructure — confirm both before reaching for it.
- Always state a DP's state in a full sentence before writing any index notation — this is the actual hard part; the code is a mechanical translation afterward.
- Memoize the brute force first for a fast working solution; convert to tabulation for speed, stack-safety, and space optimization afterward.
- 0/1 vs. unbounded knapsack is one mechanical difference: whether the transition references a strictly smaller row (0/1) or allows reusing the current row (unbounded) — and for a 1D-compressed 0/1 knapsack, the capacity loop MUST iterate downward.
- LCS carries forward the best of two neighbors on a mismatch; longest common SUBSTRING resets to zero on a mismatch — a one-line difference that changes the entire problem.
- LIS's answer is $\max(dp[i])$ over all i , not $dp[n-1]$ — the longest increasing subsequence need not end at the last element.
- Interval DP is computed by increasing interval length, so every smaller sub-interval a transition needs is already available.
- Tree DP's computation order comes for free from a post-order DFS traversal.
- Bitmask DP applies when n is small (~ 20) and state includes 'which subset has been used so far' — it collapses $(n-1)!$ orderings down to $O(2^n * n)$ states.
- Digit DP's 'tight' flag is what makes counting over ranges up to 10^{18} tractable — once a digit choice goes strictly below the bound, every remaining digit becomes free.
- A DP transition needing a range max/sum over previous states can often be optimized with a prefix sum (for sums) or a monotonic deque (for sliding-window min/max), the exact same techniques from Parts III and IV, now applied inside a DP loop.

DP Family Complexity Reference

DP family	Typical states	Typical complexity
1D DP (Fibonacci-shaped)	$O(n)$	$O(n)$
2D grid DP	$O(n * m)$	$O(n * m)$
0/1 / unbounded knapsack	$O(n * W)$	$O(n * W)$ (pseudo-polynomial — check W 's actual size)
String DP (LCS, edit distance)	$O(n * m)$	$O(n * m)$
LIS	$O(n)$	$O(n^2)$ naive, $O(n \log n)$ optimized
Interval DP	$O(n^2)$ intervals	$O(n^3)$ typical ($O(n)$ split points per interval)
Tree DP	$O(n)$ nodes	$O(n)$
Bitmask DP (TSP-shaped)	$O(2^n * n)$	$O(2^n * n^2)$
Digit DP	$O(\text{digits} * \text{extra state})$	$O(\text{digits} * \text{extra state} * 10)$

Common Mistakes Recap

Mistake	Fix
Writing <code>dp[i][j] = ...</code> before stating what it means in words	Always state the state's meaning in a full sentence first
Iterating the capacity loop upward for a 1D-compressed 0/1 knapsack	Iterate downward to avoid reusing an item twice
Confusing LCS with longest common substring	Remember: substring resets to 0 on a mismatch; subsequence carries the best neighbor forward
Returning <code>dp[n-1]</code> as the LIS answer	Return <code>max(dp[i])</code> over all <code>i</code> instead
Computing interval DP states in the wrong order (e.g. by <code>i</code> instead of by length)	Always iterate by increasing interval length
Deep tree DP recursion causing a stack overflow on large trees	Convert to an explicit iterative post-order traversal for very large <code>n</code>
Using bitmask DP when <code>n</code> is too large (beyond ~ 20 - 22)	Re-check constraints; 2^n growth makes this infeasible past that range
Trying to parametrize digit DP directly on a lower bound <code>L</code>	Compute <code>countUpTo(R) - countUpTo(L-1)</code> instead

Interview-Style Questions

- 1 What two properties must a problem have for dynamic programming to apply, and why does merge sort's recursion lack one of them?
- 2 Walk through the state, transition, and base case for 0/1 knapsack, and explain precisely why the ratio-greedy approach from Part XIV fails on it.
- 3 Explain, mechanically, the one-line difference between the LCS transition and the longest common substring transition.
- 4 Why does bitmask DP make the Traveling Salesman Problem tractable for `n` up to about 20, when brute force is not?
- 5 Explain why the 'tight' flag in digit DP allows memoization to work despite `N` being astronomically large.

- 6 Give an example of a DP whose transition can be optimized using a monotonic deque, and explain why the optimization is correct.

Master Cheat Sheet — Part XV

```
// The DP design checklist, every time:
// 1) Overlapping subproblems + optimal substructure present?
// 2) State, in one sentence?
// 3) Transition, in terms of smaller states?
// 4) Base case(s)?
// 5) Order of computation (or just memoize the brute force)?
// 6) Where does the final answer live?
// 7) Complexity = states x work per transition -- fits the
constraints?

// 0/1 knapsack (space-optimized) -- capacity loop goes DOWN
for (int i = 0; i < n; i++)
    for (int w = W; w >= weight[i]; w--)
        dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);

// Unbounded knapsack -- capacity loop goes UP
for (int w = 1; w <= W; w++)
    for (int i = 0; i < n; i++)
        if (weight[i] <= w) dp[w] = max(dp[w], dp[w - weight[i]] +
value[i]);

// Interval DP -- iterate by INCREASING LENGTH
for (int len = 2; len <= n; len++)
    for (int i = 0; i + len - 1 < n; i++) {
        int j = i + len - 1;
        for (int k = i; k < j; k++) dp[i][j] = combine(dp[i][j],
dp[i][k], dp[k+1][j]);
    }

// Bitmask DP skeleton
for (int mask = 0; mask < (1 << n); mask++)
    for (int i = 0; i < n; i++)
        if (mask & (1 << i))
            for (int j = 0; j < n; j++)
                if (!(mask & (1 << j))) { /* transition to mask |
(1<<j) */ }
```

NOTE — Coming up in Part XVI

Part XVI — Mathematics and Number Theory shifts from algorithmic structure to the numeric toolkit competitive programming leans on constantly: primes and sieves, GCD/LCM, modular arithmetic and modular inverses (needed the moment a DP's answer must be reported modulo 10^9+7 — already foreshadowed by several counting-DP transitions in this Part), combinatorics, and matrix exponentiation — including revisiting Fibonacci one more time, computed in $O(\log n)$ instead of $O(n)$.